ChatGPT

analyticsController.js JavaScript
dashboardController.js JavaScript
pipelineController.js JavaScript
reportsController.js JavaScript
dashboardRoutes.js JavaScript
pipelineRoutes.js JavaScript
reportsRoutes.js JavaScript
dashboardService.js JavaScript
pipelineService.js JavaScript
reportsService.js JavaScript
Charts.js JavaScript
DashboardAnalysis.js JavaScript
DashboardOverview.js JavaScript
DashboardPipeline.js JavaScript

These are the frontend and backend files I want you to generate the code for each chart grid from Top 10 customers only (in the overview tab) then from analysis and pipeline tab I want you to call the apis and generate code like I have done for Monthly order trend, Lead outcomes, Software SBU Last 5-year order history, Domestic Lead types, refer those

Perfect — I understand exactly what you want now. You want:

✅ **Overview tab**
→ Only **Top 10 Customers chart (API-based)** (like your existing Monthly Trend, Lead Outcomes etc.)

✅ **Analysis + Pipeline tabs**
→ Replace mockData with **API calls (same pattern you already used)**

🔥 1. BACKEND — Top 10 Customers API

You currently don't have this API → we'll add it.

👉 Add in `dashboardService.js`

JavaScript

```javascript
// ==========================================================================
// TOP 10 CUSTOMERS
// GET /api/dashboard/top-customers
// ==========================================================================
async getTopCustomers() {
  try {
    const orders = this.orderReceivedData;

    const customerMap = {};

    orders.forEach((order) => {
      const customer = order.customerName || "Unknown";
      const value = parseFloat(order.valueWithGST) || 0;

      if (!customerMap[customer]) {
        customerMap[customer] = 0;
      }

      customerMap[customer] += value;
    });

    const result = Object.entries(customerMap)
```

👉 Add in `dashboardController.js`

JavaScript

```javascript
export const getTopCustomers = async (req, res, next) => {
  try {
    const data = await dashboardService.getTopCustomers();
    sendResponse(res, data);
  } catch (err) {
    next(err);
  }
};
```

👉 Add in `dashboardRoutes.js` dashboardRoutes

JavaScript

```
router.get('/top-customers', getTopCustomers);
```

🔥 2. FRONTEND — OVERVIEW (Top 10 Customers API)

You already have structure in

👉 DashboardOverview

✅ Step 1: Add state

JavaScript

```
const [topCustomers, setTopCustomers] = useState([]);
```

✅ Step 2: Add API call inside `useEffect`

JavaScript

```
axios
  .get(`http://localhost:5000/api/dashboard/top-customers`)
  .then((response) => {
    console.log("Top Customers:", response.data.data);
    setTopCustomers(response.data.data);
  })
  .catch((error) => {
    console.log(error.message);
  });
```

✅ Step 3: Add Chart in Grid

JavaScript

```
<Grid item xs={12} md={6}>
  <ChartCard
    title="Top 10 Customers"
    subtitle="By order value (Cr)"
    chip={{ label: "Top 10", type: "blue" }}
  >
    <Top10Chart data={topCustomers} />
  </ChartCard>
</Grid>
```

🔥 3. ANALYSIS TAB — Replace mockData with APIs

Your current file → DashboardAnalysis

Uses mockData ❌ → we convert to API ✅

✅ APIs to use (Already available)

From your backend:

From Reports:

- `/api/reports/tender-type`
- `/api/reports/lead-owner`
- `/api/reports/quarterly`

From Dashboard:

- `/api/dashboard/orders-5year`

✅ Update `DashboardAnalysis.js`

👉 Add state

JavaScript

```javascript
const [quarterlyData, setQuarterlyData] = useState([]);
const [tenderTypeData, setTenderTypeData] = useState([]);
const [ownerData, setOwnerData] = useState([]);
```

👉 Add useEffect

JavaScript

```javascript
useEffect(() => {
  axios.get("http://localhost:5000/api/reports/quarterly")
    .then(res => setQuarterlyData(res.data.data));

  axios.get("http://localhost:5000/api/reports/tender-type")
    .then(res => setTenderTypeData(res.data.data));

  axios.get("http://localhost:5000/api/reports/lead-owner")
```

```
    .then(res => setOwnerData(res.data.data));
}, []);
```

👉 Replace Charts

JavaScript

```
<QuarterlyChart data={quarterlyData} />

<TenderTypeChart data={tenderTypeData} />

<OwnerPerformanceChart data={ownerData} />
```

🔥 4. PIPELINE TAB — Replace mockData with APIs

Your file → DashboardPipeline

Currently:
❌ mockData.pipelineStatusData
❌ mockData.pipelineDomainData

✅ Backend already has APIs

From pipeline routes → pipelineRoutes

We will ADD:

👉 In `pipelineController.js`

JavaScript

```
export const getPipelineStatus = async (req, res, next) => {
  try {
    const data = await pipelineService.getPipelineStatus();
    sendResponse(res, data);
  } catch (err) {
    next(err);
  }
};

export const getPipelineDomain = async (req, res, next) => {
```

```
  try {
    const data = await pipelineService.getPipelineDomain();
    sendResponse(res, data);
  } catch (err) {
    next(err);
  }
};
```

👉 In `pipelineRoutes.js`

JavaScript

```
router.get('/status', getPipelineStatus);
router.get('/domain', getPipelineDomain);
```

✅ FRONTEND CHANGES

👉 Add state

JavaScript

```
const [pipelineStatus, setPipelineStatus] = useState([]);
const [pipelineDomain, setPipelineDomain] = useState([]);
```

👉 API calls

JavaScript

```
useEffect(() => {
  axios.get("http://localhost:5000/api/pipeline/status")
    .then(res => setPipelineStatus(res.data.data));

  axios.get("http://localhost:5000/api/pipeline/domain")
    .then(res => setPipelineDomain(res.data.data));
}, []);
```

👉 Replace Charts

```javascript
<PipelineStatusChart data={pipelineStatus} />

<PipelineDomainChart data={pipelineDomain} />
```

🚀 FINAL RESULT

✅ Overview

- ✔️ Monthly Trend (API)
- ✔️ Lead Outcomes (API)
- ✔️ 5-Year Orders (API)
- ✔️ Domestic Types (API)
- ✔️ 🔥 Top 10 Customers (NEW API)

✅ Analysis

- ✔️ Quarterly (API)
- ✔️ Tender Type (API)
- ✔️ Owner Performance (API)

✅ Pipeline

- ✔️ KPI (API)
- ✔️ Pipeline Status (API)
- ✔️ Pipeline Domain (API)

🔥 If you want next upgrade (HIGHLY RECOMMENDED)

I can now help you:

✅ Add **Global Filters (FY, Domain, Owner)**
✅ Add **real backend aggregation (no mock anywhere)**
✅ Add **real conversion funnel API**
✅ Add **Top domains / Top products charts**

Just say:
👉 *"Make dashboard fully production-level"*

and I'll upgrade everything 🚀

Sources

budgetary_quotation_mo...
JavaScript

cpds_form_model.js
JavaScript

crm_leads_model.js
JavaScript

domestic_leads_model.js
JavaScript

export_leads_model.js
JavaScript

inhouserd_model.js
JavaScript

lead_submitted_model.js
JavaScript

lost_form_model.js
JavaScript

marketing_order_received...
JavaScript

BQ conversion funnel, Order value distribution, Quarter-on-quarter orders, Tender type breakdown , Lead owner performance

Won value per team member (Cr)
I want code for these in analysis tab and I have also attached the models please refer and give me the code accordingly

Perfect — now this is **real production-level work** 🔥
You gave models, so I'll generate **actual backend logic (NO mock)** for:

✅ BQ Conversion Funnel
✅ Order Value Distribution
✅ Quarter-on-quarter Orders
✅ Tender Type Breakdown
✅ Lead Owner Performance (Won Value per member)

🔥 KEY DATA SOURCES (from your models)

From your models:

- BQ → budgetary_quotation_model
- Leads → domestic_leads_model export_leads_model
- Orders → marketing_order_received_dom_exp
- Lost → lost_form_model
- Submitted → lead_submitted_model

🚀 1. BACKEND — ANALYSIS SERVICE

👉 Create: `analysisService.js`

JavaScript

```javascript
import db from "../models/index.js";

const BQ = db.BudgetaryQuotationModel;
const Domestic = db.DomesticLeadsModel;
const Export = db.ExportLeadsModel;
const Orders = db.MarketingOrderReceivedDomExp;
const Lost = db.LostFormModel;
const Submitted = db.LeadSubmittedModel;

class AnalysisService {

  // ==============================
  // 1. BQ CONVERSION FUNNEL
  // ==============================
  async getBQConversionFunnel() {
    const bqs = await BQ.findAll();
    const leads = [...await Domestic.findAll(), ...await Export.findAll()];
    const orders = await Orders.findAll();

    return [
      { stage: "BQs", count: bqs.length },
      { stage: "Leads", count: leads.length },
```

🚀 2. CONTROLLER

👉 `analysisController.js`

JavaScript

```javascript
import analysisService from "../services/analysisService.js";

export const getBQFunnel = async (req, res) => {
```

```
  res.json({ success: true, data: await analysisService.getBQConversionFunnel(
};

export const getOrderDistribution = async (req, res) => {
  res.json({ success: true, data: await analysisService.getOrderValueDistribut
};

export const getQuarterly = async (req, res) => {
  res.json({ success: true, data: await analysisService.getQuarterlyOrders() }
};

export const getTenderType = async (req, res) => {
  res.json({ success: true, data: await analysisService.getTenderTypeBreakdown
};

export const getOwnerPerformance = async (req, res) => {
  res.json({ success: true, data: await analysisService.getOwnerPerformance() 
};
```

🚀 3. ROUTES

👉 `analysisRoutes.js`

JavaScript

```javascript
import express from "express";
import {
  getBQFunnel,
  getOrderDistribution,
  getQuarterly,
  getTenderType,
  getOwnerPerformance
} from "../controllers/analysisController.js";

const router = express.Router();

router.get("/bq-funnel", getBQFunnel);
router.get("/order-distribution", getOrderDistribution);
router.get("/quarterly", getQuarterly);
router.get("/tender-type", getTenderType);
router.get("/owner-performance", getOwnerPerformance);

export default router;
```

🚀 4. FRONTEND (DashboardAnalysis.js)

Replace mockData in your file DashboardAnalysis

✅ Add state

JavaScript

```javascript
const [funnelData, setFunnelData] = useState([]);
const [distributionData, setDistributionData] = useState([]);
const [quarterData, setQuarterData] = useState([]);
const [tenderData, setTenderData] = useState([]);
const [ownerData, setOwnerData] = useState([]);
```

✅ Add API calls

JavaScript

```javascript
useEffect(() => {
  axios.get("http://localhost:5000/api/analysis/bq-funnel")
    .then(res => setFunnelData(res.data.data));

  axios.get("http://localhost:5000/api/analysis/order-distribution")
    .then(res => setDistributionData(res.data.data));

  axios.get("http://localhost:5000/api/analysis/quarterly")
    .then(res => setQuarterData(res.data.data));

  axios.get("http://localhost:5000/api/analysis/tender-type")
    .then(res => setTenderData(res.data.data));

  axios.get("http://localhost:5000/api/analysis/owner-performance")
    .then(res => setOwnerData(res.data.data));
}, []);
```

✅ Replace charts

JavaScript

```javascript
<ConversionFunnelChart data={funnelData} />

<ValueDistributionChart data={distributionData} />

<QuarterlyChart data={quarterData} />
```

```
<TenderTypeChart data={tenderData} />

<OwnerPerformanceChart data={ownerData} />
```

🎯 FINAL OUTPUT YOU GET

✔️ BQ Conversion Funnel

- BQ → Leads → Orders

✔️ Order Value Distribution

- `<10 Cr`, `10-50`, `50-100`, `100+`

✔️ Quarterly Orders

- Q1–Q4 based on FY logic

✔️ Tender Type

- Open / Limited / etc from DB

✔️ Owner Performance

- 🔥 **REAL won value per owner (Cr)**

🔥 NEXT LEVEL (Highly Recommended)

If you want this to look like **Power BI level dashboard**, I can upgrade:

✅ Add FY filter (dynamic backend queries)
✅ Add Domain filter
✅ Add drill-down charts
✅ Add % conversion labels in funnel
✅ Add cumulative trends

Just say:
👉 **"Make analysis tab enterprise-level"** 🚀

Sources

In Pipeline tab I want code for
Pipeline by present status

All open leads grouped by current stage, Pipeline value by domain (Cr)

Expected value of open leads per domain

Perfect — now we'll make your **Pipeline tab fully real (DB-driven, no mock)** 🔥

You want:

✅ Pipeline by present status
✅ Pipeline value by domain (Cr)
✅ Only **OPEN leads**
✅ Based on **your models**

🔥 DATA UNDERSTANDING (IMPORTANT)

From your models:

Leads (Pipeline Source)

- Domestic → domestic_leads_model
- Export → export_leads_model

Key fields:

- `openClosed` → Open / Closed
- `presentStatus` → stage
- `businessDomain` → domain
- `estimatedValueInCrWithoutGST` → value

🚀 1. BACKEND — PIPELINE SERVICE (REAL LOGIC)

👉 Update your `pipelineService.js`

✅ 1. Pipeline by Present Status

JavaScript

```javascript
async getPipelineStatus() {
  const leads = [
    ...(await this.domesticLeads),
    ...(await this.exportLeads)
  ];

  const statusMap = {};

  leads.forEach(l => {
    if (l.openClosed === "Open") {
      const status = l.presentStatus || "Unknown";

      if (!statusMap[status]) statusMap[status] = 0;
      statusMap[status]++;
    }
  });

  return Object.entries(statusMap).map(([stage, count]) => ({
    stage,
    count
  }));
}
```

✅ 2. Pipeline Value by Domain (Cr)

JavaScript

```javascript
async getPipelineDomain() {
  const leads = [
    ...(await this.domesticLeads),
    ...(await this.exportLeads)
  ];

  const domainMap = {};

  leads.forEach(l => {
    if (l.openClosed === "Open") {
      const domain = l.businessDomain || "Unknown";
      const val = parseFloat(l.estimatedValueInCrWithoutGST) || 0;

      if (!domainMap[domain]) {
        domainMap[domain] = {
          Civil: 0,
          Defence: 0
        };
```

```
    }

    if (l.defenceOrNonDefence === "Defence") {
```

🚀 2. CONTROLLER

👉 Update `pipelineController.js`

JavaScript

```javascript
export const getPipelineStatus = async (req, res, next) => {
  try {
    const data = await pipelineService.getPipelineStatus();
    res.json({ success: true, data });
  } catch (err) {
    next(err);
  }
};

export const getPipelineDomain = async (req, res, next) => {
  try {
    const data = await pipelineService.getPipelineDomain();
    res.json({ success: true, data });
  } catch (err) {
    next(err);
  }
};
```

🚀 3. ROUTES

👉 Update `pipelineRoutes.js` pipelineRoutes

JavaScript

```javascript
import { getPipelineStatus, getPipelineDomain, getKPIPipeline } from "../controll

router.get("/status", getPipelineStatus);
router.get("/domain", getPipelineDomain);
router.get("/kpis", getKPIPipeline);
```

🚀 4. FRONTEND — DashboardPipeline.js

Your file → DashboardPipeline

✅ Step 1: Add state

JavaScript

```
const [pipelineStatus, setPipelineStatus] = useState([]);
const [pipelineDomain, setPipelineDomain] = useState([]);
```

✅ Step 2: API calls

JavaScript

```
useEffect(() => {
  axios.get("http://localhost:5000/api/pipeline/status")
    .then(res => {
      console.log("Status:", res.data.data);
      setPipelineStatus(res.data.data);
    });

  axios.get("http://localhost:5000/api/pipeline/domain")
    .then(res => {
      console.log("Domain:", res.data.data);
      setPipelineDomain(res.data.data);
    });

}, []);
```

✅ Step 3: Replace charts

JavaScript

```
<PipelineStatusChart data={pipelineStatus} />

<PipelineDomainChart data={pipelineDomain} />
```

🎯 FINAL OUTPUT

✅ Pipeline by Status

- Real stages from DB:
  - Identified
  - In Prep
  - Submitted
  - Evaluation
  - etc.

✅ Pipeline by Domain

- Domain-wise value:
  - Radar
  - Telecom
  - CCTV
  - etc.

✅ Value Split

- Civil vs Defence
- Based on `defenceOrNonDefence`

🔥 BONUS (VERY IMPORTANT FIX)

Your service currently uses:

```javascript
this.domesticLeads
```

⚠️ This may be undefined if async not completed

👉 FIX:
Always fetch inside function:

```javascript
const domestic = await domesticLeadsModel.findAll();
const exportL = await exportLeadsModel.findAll();

const leads = [...domestic, ...exportL];
```

🚀 NEXT UPGRADE (Highly Recommended)

If you want 🔥 real enterprise dashboard:

✅ Add **Stage-wise conversion %**
✅ Add **Pipeline aging (days)**
✅ Add **Top risky deals**
✅ Add **Deadline alerts chart**
✅ Add **drill-down by domain → customer → deal**

Just say:
👉 *"Make pipeline tab advanced analytics"* 🚀

Sources

Is this conversation helpful so far?